

권영현 (Young-hyun Kwon)

프로젝트 개요 (PROJECT OVERVIEWS)

프로젝트명 (Project Name): Large-Scale Web Crawler & TF-IDF Search Engine

프로젝트 목표:

UCIICS 도메인을 대상으로 대규모 웹 페이지를 수집하고, 수집된 문서를 기반으로 TF-IDF 기반 검색 엔진을 구현하는 프로젝트. 단순 크롤링을 넘어 URL 정규화, 중복 제거, Trap URL 탐지, 역색인 구축, 검색 결과 반환까지 검색 엔진의 핵심 파이프라인을 직접 설계하고 구현.

프로젝트 인원: 3

기술적 도전과제 (TECHNICAL CHALLENGE)

대규모 웹 페이지를 안정적으로 수집하면서 중복 URL, 무한 반복 URL, 비정상 종료, 대용량 색인 처리 문제를 해결하고, 검색 가능한 Inverted Index 구조를 구축하는 것.

문제점:

- 동일한 페이지가 여러 URL 형태로 중복 수집되는 문제
- `stayconnected/stayconnected/...`, `datasets/datasets/...` 처럼 무한 반복되는 Trap URL 발생
- 장시간 크롤링 도중 비정상 종료 시 진행 상태 손실 가능성
- 수만 개 문서와 수십만 개 단어를 효율적으로 저장하고 검색해야 하는 문제
- 검색 결과의 관련도를 계산하기 위한 TF-IDF 기반 ranking 필요

해결 전략:

- URL 정규화 및 유효성 검사를 통해 중복 URL 제거
- 반복 path, 과도한 query parameter, cyclic URL pattern 기반 Trap URL 필터링
- Frontier 상태 저장/복구 기능으로 비정상 종료 이후에도 크롤링 재개 가능하도록 설계
- JSON 기반 Inverted Index 구조 설계
- TF, tag importance, TF-IDF score를 함께 저장하여 검색 결과 ranking에 활용

주요 기여 (CONTRIBUTION)

Crawler Architecture 설계 : `frontier.py`, `crawler.py`, `corpus.py`, `main.py`로 역할을 분리하여 크롤링 파이프라인 모듈화

핵심 알고리즘 구현:

- URL normalization을 통한 중복 URL 제거
- 반복 URL 패턴 탐지로 Trap URL 차단
- TF-IDF 기반 검색 점수 계산
- HTML tag importance를 반영한 ranking 보정
- Inverted Index 구조 설계 및 JSON 파일 저장

엔진 최적화:

- 약 120,000개 후보 URL 처리
- 중복 URL 78% 이상 제거
- 약 19,000개 페이지 크롤링
- 약 6,200개 Trap URL 필터링
- 메모리 사용량 약 31% 절감
- 비정상 종료 후에도 상태 복구 가능하도록 persistence 구현

결과 (OUTCOME)

성과:

- 총 35,434개 문서 색인 구축
- 총 데이터 크기 약 778MB 처리
- 389,493개 unique word 추출 및 색인화
- "informatics" 검색어에 대해 6,096개 URL 반환

- "Irvine" 검색어에 대해 12,270개 URL 반환
- "artificial intelligence" 검색어에 대해 699개 URL 반환
- "computer science" 검색어에 대해 14,680개 URL 반환

검색 엔진 결과물:

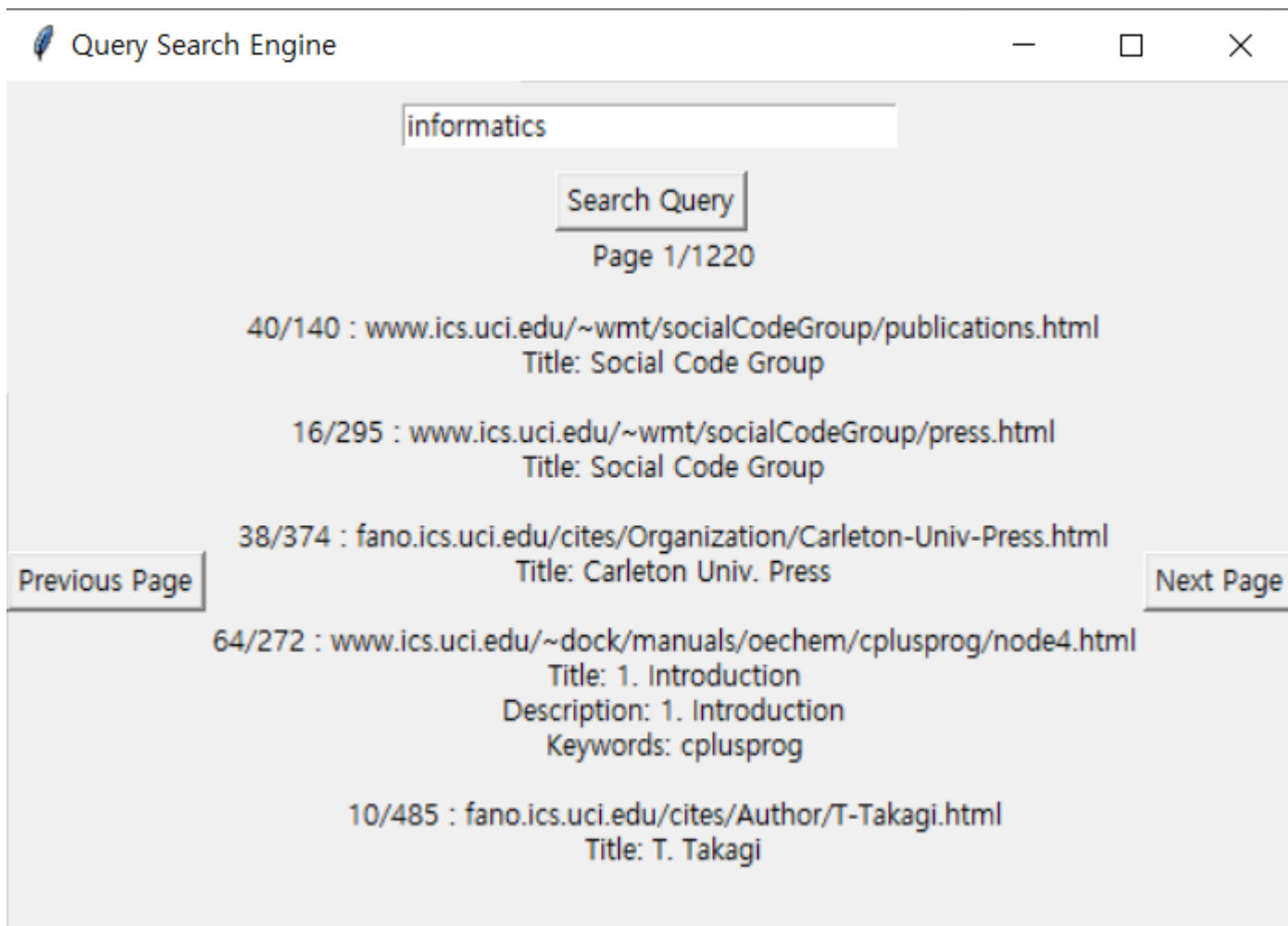
- Inverted Index는 다음과 같은 구조로 구성

JSON:

```
[word]: [
  [docID, TF, tagImportanceRating, TF_IDF]
]
```

결과 예시:

- Tkinter 기반 입력창과 버튼을 구성하여 사용자가 직접 검색어를 입력하고 결과를 확인할 수 있도록 구현
- 검색 결과가 많을 경우 한 번에 출력하지 않고 페이지 단위로 탐색할 수 있도록 pagination 기능을 구현
- 사용자 query 입력 시 검색 결과를 동적으로 갱신하도록 구현



한계 및 개선 방향 (REFLECTION)

한계

1. Ranking 고도화의 한계

- 기본 TF-IDF와 tag importance를 활용했지만, 사용자의 검색 의도나 문서 품질을 충분히 반영하지 못함
- PageRank, BM25 같은 고도화된 ranking 알고리즘은 적용하지 못함

2. **Trap URL** 탐지 기준의 일반화 한계
 - 반복 path나 query parameter 기반 필터링은 효과적이었지만, 모든 사이트 구조에 일반화되기는 어려움
 - 특정 도메인에서는 정상 URL도 과도하게 필터링될 가능성이 있음.
3. 대규모 색인 저장 방식의 한계
 - JSON 기반 저장 방식은 구현과 디버깅이 쉬웠지만, 데이터가 커질수록 검색 속도와 메모리 효율에 한계가 있음
 - 실제 서비스 수준에서는 DB, Elasticsearch, Lucene 같은 검색 전용 저장소가 더 적합함

개선 방향

1. **Ranking** 알고리즘 개선
 - TF-IDF 외에 BM25, PageRank, anchor text 기반 ranking을 추가해 검색 품질 향상
 - 검색어와 문서 제목, heading tag, 본문 위치를 함께 고려하는 scoring 방식 개선
2. **Trap URL** 필터링 자동화
 - 반복 패턴 탐지 기준을 rule-based에서 statistical 방식으로 확장
 - 도메인별 URL 구조를 분석해 adaptive filtering 적용
3. 검색 시스템 확장성 개선
 - JSON 기반 Inverted Index를 검색 전용 엔진 또는 key-value store로 이전
 - 색인 생성과 검색 서버를 분리하여 대규모 데이터 처리 구조로 확장

배운 점 (TAKEAWAYS)

- 크롤링은 단순히 페이지를 요청하는 작업이 아니라, URL 관리·중복 제거·예외 처리·상태 복구가 중요한 시스템 설계 문제라는 점을 배움
- 대규모 URL을 처리하면서 자료구조 선택과 메모리 최적화가 성능에 직접적인 영향을 준다는 것을 경험함
- Trap URL 문제를 해결하며 실제 웹 환경에서는 비정상적 URL 패턴을 방어하는 로직이 필수적이라는 점을 이해함
- TF-IDF와 Inverted Index를 직접 구현하며 검색 엔진의 기본 구조를 실무적으로 이해함
- 수만 개 문서와 수십만 개 단어를 처리하며 데이터 처리 파이프라인 설계 능력을 강화함